XPolicyLab: A Unified Standard and Open Ecosystem for Robot Policy Evaluation and Deployment

XPolicyLab Contributors, Leading Institutions: MMLab@HKU & THU
Website: https://xpolicylab.github.io/

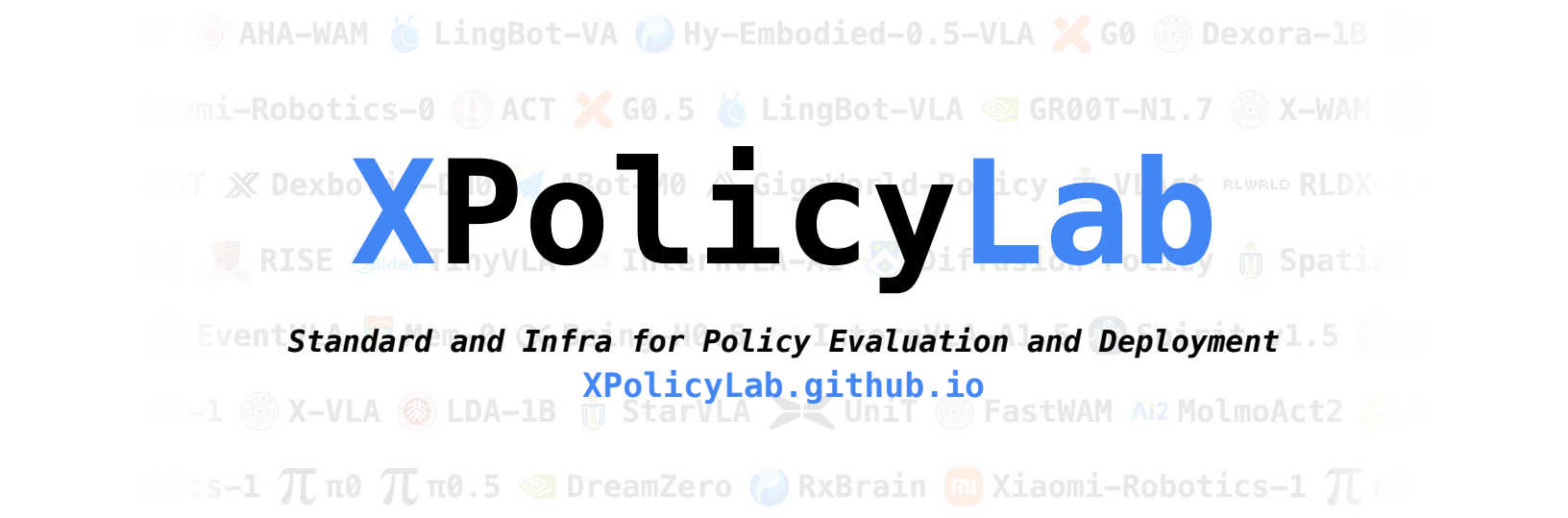


Fig. 1: **Overview of XPolicyLab.** A standard and infrastructure for robot policy evaluation and deployment.

***Abstract*— Robot policy evaluation and deployment remain fragmented by model-specific software dependencies, data representations, and runtime interfaces, so that connecting N policies to M evaluation environments requires $O(NM)$ separate integrations. We present XPolicyLab, a unified standard and open ecosystem that reduces this cost to $O(N+M)$. XPolicyLab specifies common observation, action, and trajectory schemas together with a minimal adapter interface for observation updates, action prediction, batched execution, and episode reset, while a dependency-isolated client/server architecture separates policy inference from environment execution, so that each side retains its native software stack and may run locally or remotely. The ecosystem integrates 42 robot policies and standardizes their installation, debugging, serving, and evaluation workflows. Across these adapters, model-specific code varies by an order of magnitude while the environment-facing loop stays within a few lines of a fixed reference, confirming that the contract confines heterogeneity to the policy side. In a controlled study, conforming to the standard reduces the integration effort of a representative policy from over five hours to two hours, and packaged agent skills reduce it further to thirty minutes. The same adapters serve RoboTwin, RoboDojo simulation, and standardized real-robot evaluation through one interface. XPolicyLab is released as shared infrastructure for reproducible policy comparison and standardized deployment across simulation and physical platforms. Project website: https://xpolicylab.github.io/.**

I. Introduction

Generalist robot policies, including vision-language-action (VLA) models and other embodied foundation models, have rapidly expanded in capability and task coverage [1], [2], [3], [4], [5], [6], [7], [8]. Their evaluation and deployment, however, remain constrained by fragmented system stacks [9]. Different policies assume incompatible software dependencies, observation and action representations, checkpoint conventions, and runtime interfaces, so deploying a policy on a new benchmark, simulator, or physical robot requires substantial platform-specific engineering. Observation preprocessing, action conversion, control loops, and runtime orchestration are repeatedly reimplemented, which increases integration effort and, more damagingly, silently re-fixes conventions such as camera naming, channel order, and gripper scaling, so that reported results become difficult to compare across policies. Stated as a systems problem, connecting N policies to M evaluation environments currently costs $O(NM)$ separate integrations.

Substantial progress has been made in robot data collection, dataset standardization, and training infrastructure [10], [11], [12], [13], [14], which lowers the cost of producing a checkpoint. The complementary boundary, between a trained policy and the environments that must execute it, remains largely unaddressed. A reusable solution there must absorb heterogeneous model architectures and dependencies while exposing a stable interface to simulators, benchmark clients, and physical robots. In particular, it must support stateful policies, action chunking, batched evaluation, local and remote execution, and reliable communication, without coupling policy implementations to environment-specific software stacks.

We present **XPolicyLab**,[1] a unified standard and open ecosystem for robot policy evaluation and deployment, illustrated in Fig. 2. XPolicyLab specifies common observation, action, and trajectory representations, together with a lightweight adapter contract for policy initialization, observation updates, action prediction, batched execution, and episode reset, so that each integrated policy retains its native implementation, dependencies, checkpoints, and training procedure. A dependency-isolated serving architecture separates policy inference from environment execution through a common communication layer, allowing policy servers and environment clients to run in independent software environments, either on the same machine or across a network. Around this core, XPolicyLab standardizes the workflows for installation, data conversion, training, serving, debugging, and evaluation, and packages the conformance procedure as machine-readable agent skills that a coding agent executes to scaffold, implement, and validate a new adapter.

As of August 2026, XPolicyLab integrates **42 robot policies**, spanning VLA models, world-action models, diffusion-based visuomotor policies, memory-augmented policies, and conventional imitation-learning baselines. By separating model-facing adapters from environment-facing clients, a single adapter is reused across benchmark ecosystems, simulation backends, and physical robot evaluation systems without modifying the underlying policy implementation. We demonstrate this portability through deployments on RoboTwin [15] and RoboDojo [16], including standardized physical evaluation through RoboDojo-RealEval, for which XPolicyLab also provides the policy submission and execution workflow of the official public leaderboards.

Our main contributions are summarized as follows:

1) We introduce a unified contract between robot policies and evaluation environments that standardizes observation and action representations, action prediction, batched execution, and episode management while preserving policy-specific implementations.
2) We develop a dependency-isolated serving architecture that decouples policy inference from environment execution and supports local and remote deployment across simulators, benchmark clients, and physical robot systems.
3) We package the conformance procedure as machine-readable agent skills, so that scaffolding, implementing, and auditing a new adapter against the standard becomes a guided and checkable procedure rather than a documentation-reading exercise.
4) We establish an open ecosystem comprising **42 robot policies** across diverse policy families, and evaluate it in terms of policy coverage, the locality of integration complexity, integration effort, and adapter reuse, with deployments on RoboTwin [15] and RoboDojo [16].

[1] https://github.com/XPolicyLab/XPolicyLab

II. Related Work

A. Robot Data and Training Infrastructure

Large-scale datasets and open-source frameworks have improved the availability of robot demonstrations and the reproducibility of policy training. DROID and BridgeData provide diverse real-world manipulation data [12], [17], FastUMI supports embodiment-agnostic data collection [11], LeRobot standardizes dataset processing and policy training [10], and RLinf provides scalable infrastructure for policy optimization and execution [14]. These systems target data acquisition, dataset organization, and model optimization, and therefore lower the cost of producing a checkpoint. XPolicyLab addresses the complementary boundary of how an already-trained policy is integrated, served, evaluated, and deployed across heterogeneous environment stacks.

B. Robot Policy Evaluation Platforms

Simulation benchmarks such as RLBench, ManiSkill2, LIBERO, RMBench, and UniVTAC provide controlled environments for evaluating robotic skills and generalization [18], [19], [20], [21], [22]. RoboTwin extends this setting to diverse bimanual manipulation tasks [23], [15], [24], while RoboDojo combines capability-oriented simulation with standardized physical evaluation [16]. Beyond simulation, RoboArena and RoboChallenge coordinate distributed real-robot evaluation across institutions and hardware [25], [26]. These platforms define task semantics, environment interfaces, evaluation protocols, and performance metrics; that is, they specify what is evaluated and under which conditions. XPolicyLab does not replace them, and instead supplies the policy-side abstraction that lets heterogeneous policies reach any of them through one interface, so that no benchmark needs to maintain its own model-specific serving stack.

C. Robot Policy Deployment

Robot policies such as OpenVLA, π_0, and GR00T ship inference and deployment code tailored to their respective architectures, dependencies, observation formats, and action representations [1], [27], [28]. Each such stack is faithful to one model, which is precisely why deploying a policy across multiple benchmarks, simulators, and physical robots requires repeated adaptation of preprocessing pipelines, action conversion, runtime environments, and control loops, and why the community pays the $O(NM)$ cost identified above. XPolicyLab removes this pairwise dependency through standardized observation and action schemas, a minimal adapter interface, and dependency-isolated serving. Unlike a unified modeling framework, it prescribes no network architecture, training objective, action decoder, or temporal horizon, which is what allows adapters to be reused across evaluation environments while model-specific implementations remain intact.

III. XPolicyLab

XPolicyLab adopts a policy-centric abstraction that decouples policy computation from environment execution. Its central principle is that each policy exposes a stable external contract while retaining control over its architecture,

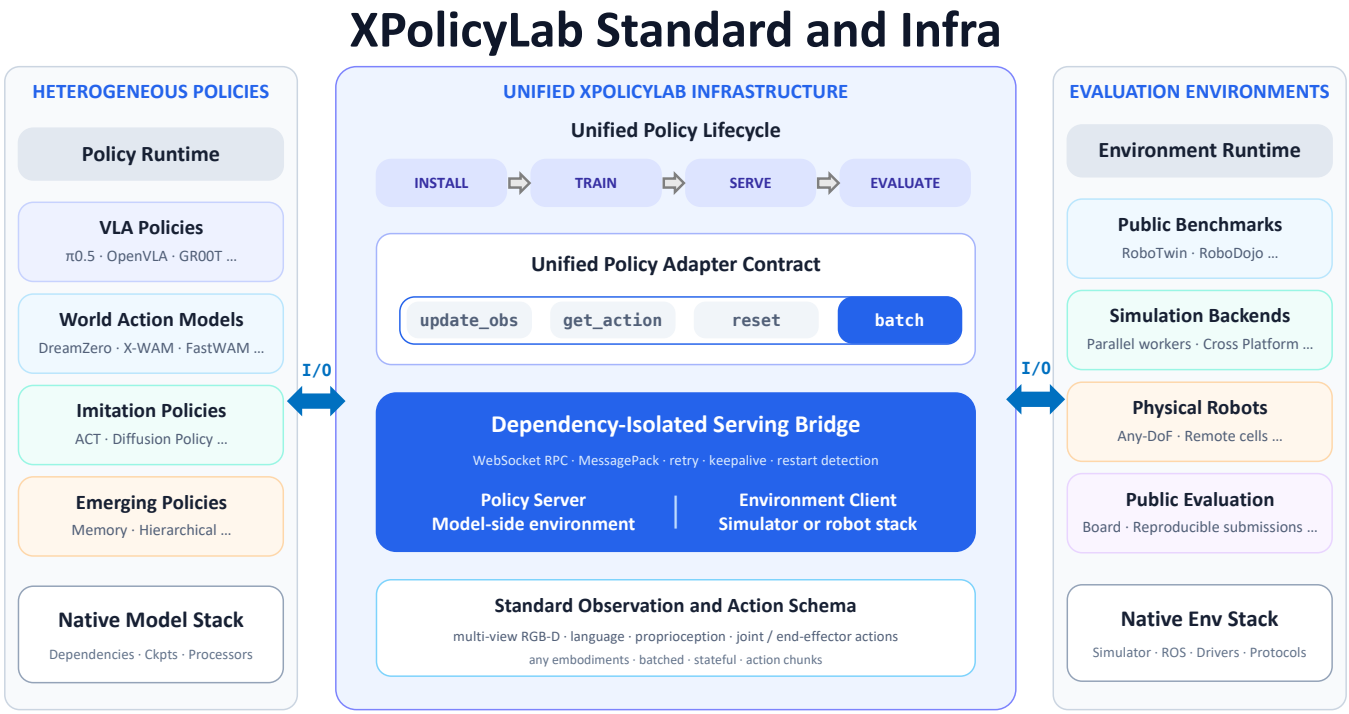


Fig. 2: **Infrastructure of XPolicyLab.** Heterogeneous policy runtimes (left) keep their native dependencies, checkpoints, and processing pipelines, while evaluation environments (right) keep their simulator and robot stacks. XPolicyLab (center) joins the two through a unified lifecycle from installation to evaluation, a minimal adapter contract, standardized observation and action schemas, and a dependency-isolated serving bridge. A single adapter therefore serves public benchmarks, simulation backends, and physical robots, under local or remote execution and batched, stateful inference.

dependencies, checkpoints, and inference procedure, so that heterogeneous benchmarks, simulators, and physical robots can invoke any policy through a common execution pathway.

A. System Overview

XPolicyLab comprises four functional components: an environment backend that executes tasks, an environment client that manages the interaction loop, a communication layer that transfers observations and actions, and a policy server that hosts the model and its adapter. Cross-cutting workflow tooling standardizes installation, configuration, debugging, training, serving, and evaluation. The policy server executes in an independent software environment and may reside either on the same machine as the environment client or on a remote compute node.

The architecture establishes two stable boundaries. A *semantic boundary* specifies the shared observation and action schemas, whereas an *execution boundary* defines communication between policy servers and environment clients. For an observation $\mathbf{o}_t$, an optional internal state $\mathbf{s}_t$, and an action horizon H, policy inference can be expressed as

$$\mathbf{a}_{t:t+H-1} = g_{\text{out}}\left(\pi_\theta\left(g_{\text{in}}(\mathbf{o}_t), \mathbf{s}_t\right)\right), \tag{1}$$

where g_{in} and g_{out} map between the shared schema and the policy-native representations. XPolicyLab standardizes these external mappings without constraining the internal forms of π_θ, g_{in}, or g_{out}.

B. Policy Adapter Contract

A policy adapter implements four core operations. Model construction loads the checkpoint, processors, and runtime configuration. `update_obs` receives a standardized observation and updates the policy context, while `get_action` returns an action chunk under the shared schema. `reset` clears episode-specific state, which is necessary for recurrent policies, temporal aggregation methods, and models with explicit memory. The corresponding `update_obs_batch` and `get_action_batch` operations support parallel evaluation and align predictions with the currently active environment instances.

The contract deliberately leaves the network architecture, action decoder, prediction horizon, and training framework unspecified, so that each adapter only translates between the shared interface and the policy's native inference API. The same evaluation loop therefore accommodates autoregressive VLA models, diffusion-based visuomotor policies, world-action models, and conventional imitation-learning methods. Integration follows a common lifecycle from installation and data preparation to configuration, serving, and evaluation, with data conversion and training omitted for declared evaluation-only integrations that ship pretrained checkpoints.

C. Dependency-Isolated Serving

Robot policies frequently depend on software stacks that conflict with simulator or robot-control environments.

XPolicyLab addresses this through process-level isolation: the policy server loads the model in its native runtime, while the environment client remains in the software environment required by the benchmark, simulator, or robot driver. The two processes communicate over a WebSocket protocol with MessagePack serialization extended for arrays, using the message set `HELLO`, `PREPARE_CASE`, `RESET`, `INFER`, `CALL`, `TRIAL_END`, `HEARTBEAT`, and `CLOSE`, each paired with an acknowledgement or result frame.

Evaluation follows the stateful loop `reset` $\rightarrow$ (`observe` $\rightarrow$ `update_obs` $\rightarrow$ `get_action` $\rightarrow$ `execute`)*. For action-chunking policies, the environment client determines how many predicted actions to execute before requesting a new chunk, which keeps that decision on the side that knows the control rate. The same protocol carries batched inference over multiple environment instances and distributed execution, allowing inference to run on a dedicated GPU server while simulation or physical control remains close to the environment.

Reliability is part of the contract, because a naive retry corrupts policy state. Requests carry unique identifiers and completed responses are cached, so a retry issued after a transient reconnection returns the cached result instead of running inference a second time; retries of a logical request therefore reuse its identifier. Each server reports an instance identifier during the handshake, and a change of that identifier is treated as fatal to the current trial, because a restarted server has lost the episode state on which the trajectory depends. Configurable connection budgets, keepalive checks, and inference timeouts accommodate long checkpoint loading times and variable inference latency.

D. Standardized Observation and Action Schemas

XPolicyLab represents an observation as

$$\mathbf{o}_t = \{\mathbf{v}_t, \mathbf{q}_t, \mathbf{p}_t, \ell, \mathbf{m}_t\}, \tag{2}$$

where $\mathbf{v}_t$ denotes visual inputs, $\mathbf{q}_t$ robot joint states, $\mathbf{p}_t$ Cartesian poses, ℓ the language instruction, and $\mathbf{m}_t$ optional metadata. Visual inputs are indexed by camera name and may contain RGB images, depth maps, and camera calibration parameters. Robot states support single-arm and bimanual embodiments, including arm joints, gripper states, and end-effector poses, and Cartesian poses follow the convention $[x, y, z, q_w, q_x, q_y, q_z]$. All fields are optional except those required by the evaluated policy.

Actions follow the same embodiment-aware organization and support both joint-space and end-effector-space control. Robot-specific action dimensions are stored in embodiment configurations rather than hard-coded in policy adapters, which preserves a stable external schema across heterogeneous kinematic structures and control spaces.

Data-processing responsibilities are divided explicitly, and this division is load bearing. The serving layer performs transport-level operations, including deserialization and image decoding, before observations reach the adapter. The adapter performs only model-specific transformations, such as resizing, normalization, tokenization, temporal-context construction, and conversion from native predictions to standardized actions. Fixing image decoding on the serving side means every adapter receives images in the same color order and layout, which removes a class of silent faults in which a policy is trained under one convention and evaluated under another.

E. Integration and Conformance

XPolicyLab treats policy integration as conformance to a shared specification rather than as environment-specific implementation work. A policy package provides a runtime configuration, model adapter, execution scripts, documentation, and a reproducible checkpoint specification, with data-conversion and training entry points included when available.

Adapters are validated progressively, cheapest gate first. Static checks verify configuration, imports, and script integrity. An offline closed-loop client then exercises server startup, observation serialization, action structure, batched execution, and episode reset to a deterministic finish marker, without requiring a simulator or physical robot. Only after these interface-level checks pass is the adapter connected to a supported environment.

Conversely, a new environment only needs to construct standardized observations, invoke the policy operations, and execute the returned actions, with policy-specific inference logic remaining behind the serving boundary. Extending the ecosystem therefore requires one adapter per policy or one client per environment, rather than separate integration code for every pairing of policy and environment.

F. Agent-Assisted Integration

Because the conformance procedure is fully specified, it can be executed rather than read. The workflow is packaged as machine-readable *agent skills* shipped with the repository: an integration skill that encodes the adapter layout, contract rules, checkpoint conventions, and the debug closed-loop procedure, and an audit skill that checks a submission against the same gates before review. Coding agents such as Cursor, Claude Code, and Codex load these skills to scaffold, implement, and validate a new adapter under human supervision, turning policy integration from a documentation-reading exercise into a guided and checkable procedure. Section VI-A quantifies the effect.

IV. Policy Ecosystem

A central objective of XPolicyLab is to support heterogeneous robot policies without constraining their architectures, training objectives, or inference procedures. As of August 8, 2026, XPolicyLab integrates **42 robot policies**, summarized in Table I: VLA models, world-action models, diffusion-based visuomotor policies, memory-augmented policies, and conventional imitation-learning baselines.

Each integration encapsulates model-specific dependencies, checkpoint loading, observation preprocessing, action decoding, and state management behind the common contract, so that environment clients invoke every supported policy through the same observation-update, action-prediction,

TABLE I: Robot policies supported by XPolicyLab as of August 8, 2026. Links point to the corresponding adapter implementations.

A1 [29]	AHA-WAM [30]	ABot-M0 [31]	ACT [32]	Being-H0.5 [33]	Dexbotic-DM0 [34]
Dexora-1B [35]	Diffusion Policy [36]	DreamZero [5]	EventVLA [37]	FastWAM [38]	G0.5 [39]
G0 [40]	GigaWorld-Policy [41]	GO-1 [42]	GR00T-N1.7 [28]	H-RDT [43]	Hy-Embodied-0.5-VLA [7]
InternVLA-A1 [44]	InternVLA-A1.5 [45]	LDA-1B [46]	LingBot-VA [47]	LingBot-VLA [48]	Mem-0 [21]
MolmoAct2 [49]	OpenVLA-OFT [1]	π_0 [27]	$\pi_{0.5}$ [2]	VLAct [50]	RDT-1B [51]
RISE [52]	SmolVLA [53]	Spatial Forcing [54]	Spirit v1.5 [55]	StarVLA [13]	TinyVLA [56]
X-VLA [57]	X-WAM [58]	Xiaomi-Robotics-0 [6]	Xiaomi-Robotics-1 [59]	UniT [60]	RxBrain [61]

batched-execution, and episode-reset operations. All adapters expose the standardized serving and evaluation interface, while data-conversion and training entry points are included when the corresponding implementations are publicly available.

The ecosystem also spans heterogeneous temporal and computational requirements, from single-step prediction and action chunking to temporal aggregation and explicit memory, and from lightweight imitation-learning baselines to large foundation models that require an isolated GPU runtime. Whether this breadth is genuinely absorbed by the contract, rather than merely tolerated, is an empirical question; Section VI-A answers it by measuring the effort that integration actually costs.

V. Applications and Leaderboard Evidence



Fig. 3: **Cross-platform policy evaluation through XPolicyLab.** A shared policy codebase and a standardized serving interface allow the same policy integration to be evaluated in RoboTwin 2.0, RoboDojo simulation, and RoboDojo real-world settings with minimal policy-side adaptation.

We examine XPolicyLab through its deployment in two public benchmark ecosystems covering three evaluation settings (Fig. 3): RoboTwin simulation, RoboDojo simulation, and RoboDojo real-world evaluation. These settings differ in task definitions, simulator infrastructure, robot embodiments, and evaluation protocols, yet share the same policy abstraction and serving architecture. The leaderboards therefore characterize the operational coverage and portability of XPolicyLab rather than algorithmic improvements, and detailed task definitions and policy-level analyses belong to the corresponding benchmark papers [15], [16].

A. RoboTwin Simulation

RoboTwin evaluates bimanual manipulation across 50 tasks under clean and randomized settings [15]. XPolicyLab connects policies to RoboTwin through an environment client that constructs standardized observations and executes returned actions, while policy adapters remain responsible for model-specific preprocessing, checkpoint loading, internal state management, and action decoding. This separation allows RoboTwin to reuse one serving stack across policies with heterogeneous architectures and dependencies.

Table II reports the top 10 policies ranked by mean success rate under the clean setting, with the randomized column characterizing performance under the more challenging configuration. All listed policies completed evaluation on all 50 tasks under both settings. The live leaderboard is available at robotwin-platform.github.io/leaderboard.

TABLE II: Top 10 RoboTwin policies ranked by mean clean success rate. Snapshot dated Aug 10, 2026.

Rank	Policy	Clean SR (%)	Randomized SR (%)
1	FastWAM [38]	77.8	1.9
2	Spatial Forcing [54]	77.2	9.5
3	$\pi_{0.5}$ [2]	70.7	46.0
4	X-WAM [58]	70.0	25.8
5	X-VLA [57]	68.0	20.9
6	EventVLA [37]	65.6	15.7
7	AHA-WAM [30]	64.3	3.2
8	Xiaomi-Robotics-0 [6]	62.9	18.2
9	G0 [40]	62.7	9.1
10	Abot-M0 [31]	57.4	22.9

B. RoboDojo Simulation

RoboDojo provides 42 simulation tasks organized into five capability dimensions: Generalization, Precision, Long-Horizon, Memory, and Open [16]. Its environment workers execute heterogeneous tasks concurrently in Isaac Sim, while XPolicyLab policy servers perform observation updates and batched action prediction. This process separation matters most here, since many large policies carry dependencies that are incompatible with the simulator runtime.

Table III reports the top 10 policies ranked by overall score averaged across the five capability dimensions. Success rate (SR) measures complete task execution, whereas the score additionally credits partial task progress. All evaluations use the common XPolicyLab environment-facing interface and

TABLE III: Top 10 policies on the RoboDojo simulation leaderboard ranked by average score. Snapshot dated Aug 4, 2026.

Rank	Policy	Score	SR (%)
1	G0.5 [39]	20.23	14.88
2	Xiaomi-Robotics-1 [59]	20.07	13.93
3	Hy-Embodied-0.5-VLA [7]	13.07	8.80
4	Spatial Forcing [54]	12.38	8.04
5	$\pi_{0.5}$ [2]	11.41	6.91
6	InternVLA-A1.5 [45]	11.15	7.14
7	VLAct [50]	10.66	7.60
8	X-VLA [57]	10.13	6.52
9	X-WAM [58]	7.69	3.83
10	Xiaomi-Robotics-0 [6]	6.93	4.18

TABLE IV: Top 10 policies on the RoboDojo-RealEval leaderboard ranked by average score. Snapshot dated Aug 4, 2026.

Rank	Policy	Score	SR (%)
1	$\pi_{0.5}$ [2]	22.9	12.8
2	InternVLA-A1 [44]	12.0	7.2
3	G0 [40]	9.0	4.4
4	Xiaomi-Robotics-0 [6]	7.9	3.9
5	X-VLA [57]	7.6	3.3
6	GR00T-N1.7 [28]	5.9	1.7
7	π_0 [27]	5.8	1.7
8	StarVLA-α [13]	4.1	1.7
9	Spirit v1.5 [55]	1.6	0.6
10	Dexbotic-DM0 [34]	0.0	0.0

the RoboDojo evaluation protocol. The live leaderboard is available at robodojo-benchmark.com/leaderboard.

C. RoboDojo Real-World Evaluation

RoboDojo-RealEval extends the same serving architecture to 18 physical tasks across three bimanual robot embodiments: ARX X5, Piper, and Piper X [16]. The robot controller acts as the XPolicyLab environment client, while policy inference runs in an isolated process, potentially on a remote GPU server. Hardware control, sensing synchronization, scene reset, and safety supervision remain on the environment side. When the required observation and action representations are preserved, a policy adapter is reused without modifying its model-specific inference implementation.

Each policy is evaluated over 10 trials per task, yielding 180 physical trials across the three embodiments. Table IV reports the top 10 policies ranked by overall score, excluding human teleoperation.

Two observations matter for an infrastructure audience. First, absolute performance is low in both simulation and reality, and real-world rankings only partially track simulation ones, so what currently binds progress is the cost of running a physical trial rather than the cost of another simulation seed. Second, the gap between clean and randomized RoboTwin success rates differs sharply across policies, and that gap is interpretable only because all entries share one observation contract; had each submission shipped its own preprocessing, differences of this magnitude would be indistinguishable from packing discrepancies.

TABLE V: Integration effort for connecting $\pi_{0.5}$ to RoboDojo simulation (median over $N = 6$ participants). The agent condition uses Cursor with Opus 5.

Phase	From scratch	XPolicyLab (manual)	XPolicyLab + agent skills
Environment setup	2 h	30 min	10 min
Obs./action glue code	2 h	1 h	10 min
Debugging to 1st rollout	1.5 h	30 min	10 min
Total (median)	**>5 h**	**∼2 h**	**∼30 min**
Hand-written LoC	∼300	∼120	∼0

VI. System Evaluation

A. Integration and Reproduction Effort

We quantify integration cost with a controlled study on a representative task: connecting $\pi_{0.5}$ to the RoboDojo simulation evaluation and reproducing a closed-loop result. Participants ($N = 6$ engineers unfamiliar with XPolicyLab) performed the task under three conditions: (i) *from scratch*, using only the upstream model repository; (ii) through XPolicyLab, following the adapter standard manually; and (iii) through XPolicyLab with the packaged agent skills, where a coding agent executes the scaffold–implement–debug cycle under human supervision.

The agent condition fixes the agent stack so that the comparison is about the skills rather than about model choice: all participants used Cursor as the coding agent with Opus 5 as the underlying model, on the same repository checkout, with the integration and audit skills loaded from `.agents/skills/` and no additional prompting scaffold. The agent performs scaffolding, adapter implementation, and iteration against the audit gates, while the human supplies the run configuration, launches the debug closed loop, and reviews the generated adapter before accepting it.

The study is within-subject: every participant completed all three conditions. Six participants admit exactly the six orderings of three conditions, so each ordering was assigned to one participant and no condition is systematically advantaged by position. A participant's sessions were separated by at least one day, and participants worked only from the materials of the current condition, without reusing code or notes from an earlier one. Counterbalancing cannot remove residual familiarity with $\pi_{0.5}$ itself, which is one reason we read the result as an indicative case study rather than a controlled measurement of the standard in isolation. We measure time-to-first-successful-rollout and hand-written lines of code (LoC), that is, code written by a human rather than generated by the agent and reviewed, excluding checkpoint download time, which is bandwidth-bound and identical across conditions.

Table V shows that conforming to the XPolicyLab standard reduces the integration effort of a representative VLA policy

from over five hours to roughly two hours, and that the packaged agent skills reduce it further to thirty minutes, an order-of-magnitude reduction over the from-scratch baseline, because the agent executes the documented procedure while the human only configures and reviews. Reproducing the evaluation of an *already-integrated* policy is cheaper still: installation and a single evaluation command complete in $\sim$10 minutes, again excluding the bandwidth-bound checkpoint download, versus hours of environment and glue-code reconstruction without the shared layer.

VII. Conclusion

We presented **XPolicyLab**, a unified standard and open ecosystem for robot policy evaluation and deployment, combining a minimal policy adapter contract, standardized observation and action schemas, and dependency-isolated local and remote serving. Measured across its 42 adapters, the design behaves as intended: model-specific code varies by an order of magnitude while the environment-facing loop stays within a few lines of a fixed reference, and conforming to the standard reduces the integration effort of a representative policy from over five hours to two hours, and to thirty minutes once the packaged agent skills execute the conformance procedure. The same adapters drive RoboTwin, RoboDojo simulation, and RoboDojo-RealEval, so that heterogeneous policies retain their native implementations while sharing one evaluation pathway. By replacing pairwise integration between policies and environments with reusable adapters and environment clients, XPolicyLab lowers the cost of the step that currently gates reproducible comparison: getting a checkpoint to run correctly in a setting where it has never run before. The standard cannot remove physical variance or the scarcity of standardized robot time; it can only ensure that scarce time is spent on physical interaction rather than on rediscovering how to feed each checkpoint.

VIII. Contributors

XPolicyLab is a collaborative open-source project led by **MMLab@HKU** and **THU**, with project leadership provided by Tianxing Chen. The complete list of contributors and their policy integrations is provided below. Project website: xpolicylab.github.io.

Core Lead Authors.
Tianxing Chen, Yue Chen, Tian Nian, Zijian Cai, Guangyu Chen, Wenwei Lin, Qiwei Liang.

Contributors.
Peicheng Xiang, Kailun Su, Zixuan Li, Junyuan Tang, Yan Qin, Qiangyu Chen, Shaolong Zhu, Xiang Li, Jiahao Zhang, Weijie Wan, Baijun Chen, Honghao Su, Kehe Ye, Shujia Liu and Xspark AI Team (**Benchmark Infrastructure**; $\pi_{0.5}$, **X-VLA**, **Xiaomi-Robotics-0**, **StarVLA**, **G0**, **ABot-M0**, **FastWAM**, π_0, **GR00T-N1.7**, **InternVLA-A1**, **SmolVLA**, **LDA-1B**, **MolmoAct2**, **GO-1**, **ACT**, **RDT-1B**, **DM0**, **TinyVLA**, **OpenVLA-OFT**, **RLDX-1**); Kaixuan Wang, Haotian Liang (**Hy-Embodied-0.5-VLA**, **RxBrain**); Yunze Liu (**DreamZero**); Mingleyang Li and Yuran Wang (**Mem-0**); Boyu Chen (**UniT**); Hongzhe Bi (**Motus H-RDT**); Shuhe Huang and Hengkai Tan (**MotuBrain**); Jisong Cai and Yao Mu (**AHA-WAM**); Jun Guo (**X-WAM**); Xiaofeng Wang, Zheng Zhu, Weijie Ke, and Hengtao Li (**GigaWorld-Policy**); Yuhang Tang and Xiaofan Li (**Wall-WM**, **Wall-OSS**); Ganlin Yang and Zhangzheng Tu (**EventVLA**); Shuai Yang (**LingBot-VLA**, **LingBot-VA**); Wenxuan Song and Pengxiang Ding (**Spatial Forcing**); Kaidong Zhang and Yu Sun (**A1**); Junliang Guo, Tong Zhang, and Yixing Chen (**Spirit v1.5**); Rongxu Cui and Zongzheng Zhang (**Dexora-1B**); Haoxiang Ma and Junhao Cai (**InternVLA-A1.5**); Haoyu Zhang (**G0.5**); Senqiao Yang, Jinhui Ye, Pengguang Chen, and Shu Liu (**VLAct**); Xiu Su, Wenhan Fang, Wenhao Li, Yichao Cao, Chengyao Wang, and Qiang Chen (**CSU-AI-0**); XR-1 Team (**Xiaomi-Robotics-1**).

Corresponding Authors.
Wenbo Ding and Ping Luo. For questions regarding the project, please contact ding.wenbo@sz.tsinghua.edu.cn and pluo@cs.hku.hk.

XPolicyLab is being actively maintained and expanded; we will update this report as the project evolves.

References

[1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[2] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, "$pi_{0.5}$: A vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.

[3] P. Intelligence, B. Ai, A. Amin, R. Aniceto, A. Balakrishna, G. Balke, K. Black, G. Bokinsky, S. Cao, T. Charbonnier *et al.*, "Pi 0.7: a steerable generalist robotic foundation model with emergent capabilities," *arXiv preprint arXiv:2604.15483*, 2026.

[4] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu *et al.*, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024.

[5] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang *et al.*, "World action models are zero-shot policies," *arXiv preprint arXiv:2602.15922*, 2026.

[6] R. Cai, J. Guo, X. He, P. Jin, J. Li, B. Lin, F. Liu, W. Liu, F. Ma, K. Ma *et al.*, "Xiaomi-robotics-0: An open-sourced vision-language-action model with real-time execution," *arXiv preprint arXiv:2602.12684*, 2026.

[7] H. Zhang, L. Xiang, H. Lin, Z. Huang, M. Wang, D. Zhong, Y. Dong, Y. Wu, Y. Rao, D. Zhang *et al.*, "Hy-embodied-0.5-vla: From vision-language-action models to a real-world robot learning stack," *arXiv preprint arXiv:2606.14409*, 2026.

[8] T. Chen, Y. Mu, Z. Liang, Z. Chen, S. Peng, Q. Chen, M. Xu, R. Hu, H. Zhang, X. Li *et al.*, "G3flow: Generative 3d semantic flow for pose-aware and generalizable object manipulation," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 1735–1744.

[9] X. Yang, T. Chen, H. Su, M. Wang, C. Yu, Z. Tu, Y. Chen, Y. Huo, L. Zhang, Y. Huang *et al.*, "Towards trustworthy embodied intelligence: A systems framework and graded trustworthiness levels," *arXiv preprint arXiv:2607.26121*, 2026.

[10] R. Cadene, S. Alibert, F. Capuano, M. Aractingi, A. Zouitine, P. Kooijmans, J. Choghari, M. Russi, C. Pascal, S. Palma *et al.*, "Lerobot: An open-source library for end-to-end robot learning," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 122 398–122 417.

[11] K. Liu, C. Guan, Z. Jia, Z. Wu, X. Liu, T. Wang, S. Liang, P. Chen, P. Zhang, H. Song *et al.*, "Fastumi: A scalable and hardware-independent universal manipulation interface with dataset," *arXiv preprint arXiv:2409.19499*, 2024.

[12] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis *et al.*, "Droid: A large-scale in-the-wild robot manipulation dataset," *arXiv preprint arXiv:2403.12945*, 2024.

[13] S. Community, "Starvla: A lego-like codebase for vision-language-action model developing," *arXiv preprint arXiv:2604.05014*, 2026.

[14] C. Yu, Y. Wang, Z. Guo, H. Lin, S. Xu, H. Zang, Q. Zhang, Y. Wu, C. Zhu, J. Hu *et al.*, "{RLinf}: Flexible and efficient {Large-Scale} reinforcement learning via {Macro-to-Micro} flow transformation," in *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, 2026, pp. 829–846.

[15] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu *et al.*, "Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation," *arXiv preprint arXiv:2506.18088*, 2025.

[16] T. Chen, Y. Chen, Z. Li, J. Tang, K. Su, W. Wan, B. Chen, H. Lu, H. Yan, H. Su *et al.*, "RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies," *arXiv preprint arXiv:2607.04434*, 2026.

[17] F. Ebert, Y. Yang, K. Schmeckpeper, B. Bucher, G. Georgakis, K. Daniilidis, C. Finn, and S. Levine, "Bridge data: Boosting generalization of robotic skills with cross-domain datasets," *arXiv preprint arXiv:2109.13396*, 2021.

[18] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, "Rlbench: The robot learning benchmark & learning environment," 2019. [Online]. Available: https://arxiv.org/abs/1909.12271

[19] J. Gu, F. Xiang, X. Li, Z. Ling, X. Liu, T. Mu, Y. Tang, S. Tao, X. Wei, Y. Yao *et al.*, "Maniskill2: A unified benchmark for generalizable manipulation skills," in *The Eleventh International Conference on Learning Representations*, 2023.

[20] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.

[21] T. Chen, Y. Wang, M. Li, Y. Qin, H. Shi, Z. Li, Y. Hu, Y. Zhang, K. Wang, Y. Chen *et al.*, "Rmbench: Memory-dependent robotic manipulation benchmark with insights into policy design," *arXiv preprint arXiv:2603.01229*, 2026.

[22] B. Chen, W. Wan, T. Chen, X. Guo, C. Xu, Y. Qi, H. Zhang, L. Wu, T. Xu, Z. Li *et al.*, "Univtac: A unified simulation platform for visuo-tactile manipulation data generation, learning, and benchmarking," *arXiv preprint arXiv:2602.10093*, 2026.

[23] Y. Mu, T. Chen, Z. Chen, S. Peng, Z. Lan, Z. Gao, Z. Liang, Q. Yu, Y. Zou, M. Xu *et al.*, "Robotwin: Dual-arm robot benchmark with generative digital twins," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 27 649–27 660.

[24] T. Chen, K. Wang, Z. Yang, Y. Zhang, Z. Chen, B. Chen, W. Dong, Z. Liu, D. Chen, T. Yang *et al.*, "Benchmarking generalizable bimanual

manipulation: Robotwin dual-arm collaboration challenge at cvpr 2025 meis workshop," *arXiv preprint arXiv:2506.23351*, 2025.

[25] P. Atreya, K. Pertsch, T. Lee, M. J. Kim, A. Jain, A. Kuramshin, C. Eppner, C. Neary, E. Hu, F. Ramos *et al.*, "Roboarena: Distributed real-world evaluation of generalist robot policies," *arXiv preprint arXiv:2506.18123*, 2025.

[26] A. Yakefu, B. Xie, C. Xu, E. Zhang, E. Zhou, F. Jia, H. Yang, H. Fan, H. Zhang, H. Peng *et al.*, "Robochallenge: Large-scale real-robot evaluation of embodied policies," *arXiv preprint arXiv:2510.17950*, 2025.

[27] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, "pi_0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[28] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang *et al.*, "Gr00t n1: An open foundation model for generalist humanoid robots," *arXiv preprint arXiv:2503.14734*, 2025.

[29] K. Zhang, J. Zhang, R. Xu, Y. Sun, S. Xue, Y. Wen, X. Guo, M. Guo, W. Liufu, L. Zihou *et al.*, "A1: A fully transparent open-source, adaptive and efficient truncated vision-language-action model," *arXiv preprint arXiv:2604.05672*, 2026.

[30] J. Cai, L. Ling, S. Chu, Z. Liu, J. Kang, Z. Liang, W. Xu, Y. Mao, W. Zhang, X. Yang *et al.*, "Aha-wam: Asynchronous horizon-adaptive world-action modeling with observation-guided context routing," *arXiv preprint arXiv:2606.09811*, 2026.

[31] Y. Yang, S. Zeng, T. Lin, X. Chang, D. Qi, J. Xiao, H. Liu, R. Chen, Y. Chen, D. Huo *et al.*, "Abot-m0: Vla foundation model for robotic manipulation with action manifold learning," *arXiv preprint arXiv:2602.11236*, 2026.

[32] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," *arXiv preprint arXiv:2304.13705*, 2023.

[33] H. Luo, Y. Wang, W. Zhang, S. Zheng, Z. Xi, C. Xu, H. Xu, H. Yuan, C. Zhang, Y. Wang *et al.*, "Being-h0. 5: Scaling human-centric robot learning for cross-embodiment generalization," *arXiv preprint arXiv:2601.12993*, 2026.

[34] E. Yu, H. Lv, J. Sun, K. Lin, R. Zhang, Y. Shi, Y. Chen, Z. Chen, Z. Zhang, F. Jia *et al.*, "Dm0: An embodied-native vision-language-action model towards physical ai," *arXiv preprint arXiv:2602.14974*, 2026.

[35] Z. Zhang, J. Pang, Z. Yang, K. Li, M. Liao, S. Zhang, G. Chi, J. Guo, H.-a. Gao, M. Shi *et al.*, "Dexora: Open-source vla for high-dof bimanual dexterity," *arXiv preprint arXiv:2605.18722*, 2026.

[36] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

[37] G. Yang, Z. Tu, Y. Yang, S. Mao, J. Dong, T. Chen, J. Peng, J. Xiong, J. Cao, J. Dai *et al.*, "Eventvla: Event-driven visual evidence memory for long-horizon vision-language-action policies," *arXiv preprint arXiv:2606.20092*, 2026.

[38] T. Yuan, Z. Dong, Y. Liu, and H. Zhao, "Fast-wam: Do world action models need test-time future imagination?" *arXiv preprint arXiv:2603.16666*, 2026.

[39] Galaxea Team, "Galaxea g0.5 technical report," 2026. [Online]. Available: https://opengalaxea.github.io/G05/

[40] T. Jiang, T. Yuan, Y. Liu, C. Lu, J. Cui, X. Liu, S. Cheng, J. Gao, H. Xu, and H. Zhao, "Galaxea open-world dataset and g0 dual-system vla model," *arXiv preprint arXiv:2509.00576*, 2025.

[41] A. Ye, B. Wang, C. Ni, G. Huang, G. Zhao, H. Li, H. Li, J. Li, J. Lv, J. Liu *et al.*, "Gigaworld-policy: An efficient action-centered world–action model," *arXiv preprint arXiv:2603.17240*, 2026.

[42] Q. Bu, J. Cai, L. Chen, X. Cui, Y. Ding, S. Feng, S. Gao, X. He, X. Hu, X. Huang *et al.*, "Agibot world colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems," *arXiv preprint arXiv:2503.06669*, 2025.

[43] H. Bi, L. Wu, T. Lin, H. Tan, Z. Su, H. Su, and J. Zhu, "H-rdt: Human manipulation enhanced bimanual robotic manipulation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 22, 2026, pp. 18 135–18 143.

[44] J. Cai, Z. Cai, J. Cao, Y. Chen, Z. He, L. Jiang, H. Li, H. Li, Y. Li, Y. Liu *et al.*, "Internvla-a1: Unifying understanding, generation and action for robotic manipulation," *arXiv preprint arXiv:2601.02456*, 2026.

[45] H. Ma, J. Cai, X. Xu, H. Li, Y. Yang, Y. Tian, J. Cao, H. Zhu, Z. Qiu, Y. Yang *et al.*, "Internvla-a1. 5: Unifying understanding, latent foresight, and action for compositional generalization," *arXiv preprint arXiv:2607.04988*, 2026.

[46] J. Lyu, K. Liu, X. Zhang, H. Liao, Y. Feng, W. Zhu, T. Shen, J. Chen, J. Zhang, Y. Dong *et al.*, "Lda-1b: Scaling latent dynamics action model via universal embodied data ingestion," *arXiv preprint arXiv:2602.12215*, 2026.

[47] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, F. Han, M. Yu, Z. Gao, N. Xue, X. Zhu, Y. Shen, and Y. Xu, "Causal world modeling for robot control," 2026. [Online]. Available: https://arxiv.org/abs/2601.21998

[48] W. Wu, F. Lu, Y. Wang, S. Yang, S. Liu, F. Wang, Q. Zhu, H. Sun, Y. Wang, S. Ma *et al.*, "A pragmatic vla foundation model," *arXiv preprint arXiv:2601.18692*, 2026.

[49] H. Fang, J. Duan, D. Clay, S. Wang, S. Liu, W. Huang, X. Fan, W.-C. Tsai, S. Chen, Y. R. Wang *et al.*, "Molmoact2: Action reasoning models for real-world deployment," *arXiv preprint arXiv:2605.02881*, 2026.

[50] J. Ye, N. Gao, S. Yang, J. Zheng, Z. Wang, Y. Chen, P. Chen, Y. Chen, S. Liu, and J. Jia, "Starvla-α: Reducing complexity in vision-language-action systems," *arXiv preprint arXiv:2604.11757*, 2026.

[51] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, "Rdt-1b: a diffusion foundation model for bimanual manipulation," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 29 982–30 009.

[52] J. Yang, K. Lin, J. Li, W. Zhang, T. Lin, L. Wu, Z. Su, H. Zhao, Y.-Q. Zhang, L. Chen *et al.*, "Rise: Self-improving robot policy with compositional world model," *arXiv preprint arXiv:2602.11075*, 2026.

[53] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti *et al.*, "Smolvla: A vision-language-action model for affordable and efficient robotics," *arXiv preprint arXiv:2506.01844*, 2025.

[54] F. Li, W. Song, H. Zhao, J. Wang, P. Ding, D. Wang, L. Zeng, and H. Li, "Spatial forcing: Implicit spatial representation alignment for vision-language-action model," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 132 324–132 345.

[55] S. A. Team, "Spirit-v1.5: Clean data is the enemy of great robot foundation models," *Spirit AI Blog*, 2026, https://www.spirit-ai.com/en/blog/spirit-v1-5.

[56] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen *et al.*, "Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation," *IEEE Robotics and Automation Letters*, 2025.

[57] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng *et al.*, "X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 60 580–60 606.

[58] J. Guo, Q. Li, P. Li, Z. Chen, N. Sun, Y. Su, H. Wang, Y. Zhang, X. Li, and H. Liu, "Unified 4d world action modeling from video priors with asynchronous denoising," *arXiv preprint arXiv:2604.26694*, 2026.

[59] X. R. Team, J. Guo, P. Jin, J. Li, P. Li, Y. Li, F. Liu, W. Peng, O. Qin, Y. Su *et al.*, "Xiaomi-robotics-1: Scaling vision-language-action models with over 100k hours of real-world trajectories," *arXiv preprint arXiv:2607.15330*, 2026.

[60] T. U. o. H. K. XPENG Robotics, Tsinghua University, "Unit: Toward a unified physical language for human-to-humanoid policy learning and world modeling," *https://xpeng-robotics.github.io/unit/*, 2026.

[61] H. Liang, M. Chen, Y. Huang, Y. Guo, X. Zhu, X. Shi, K. Wang, Y. Mao, W. Zhou, L. Chen *et al.*, "Rxbrain: Embodied cognition foundation model with joint language-visual reasoning and imagination," *arXiv preprint arXiv:2607.14187*, 2026.